

Spécification API

Auteur : Jonatan NSUALU

Date : Mai 2026

Introduction

- ◆ [URL de base](#)
- ◆ [Format des échanges](#)
- ◆ [Architecture REST](#)
- ◆ [Liste des endpoints principaux](#)
- ◆ [Exemple de route API](#)

Standardisation des réponses API

- ◆ [Structure d'une réponse API](#)
- ◆ [Exemple de réponse succès](#)
- ◆ [Exemple de réponse erreur](#)
- ◆ [Pourquoi utiliser un format unique ?](#)

WebSocket

Documentation API

- ◆ [Technologies utilisées](#)

Introduction

L'application ConneCTeam repose sur une API REST développée avec Spring Boot afin de permettre la communication entre le frontend Angular et le backend.

Cette API permet la gestion des utilisateurs, des tâches, des messages, des actualités et des notifications de manière sécurisée et structurée.

◆ URL de base

Durant la phase de développement, l'API sera accessible localement via l'URL suivante :

```
http://localhost:8080/api
```

Cette URL est utilisée uniquement pour l'environnement de développement.

En environnement de production, une URL différente sera utilisée afin de permettre l'accès au service depuis Internet et d'assurer la sécurisation ainsi que le déploiement de l'application.

◆ Format des échanges

Les données échangées entre le frontend et le backend utilisent le format JSON (*JavaScript Object Notation*).

```
{  
  "nom": "Dupont",  
  "prenom": "Jean"  
}
```

Le format JSON a été choisi car il est :

- léger,
- facilement lisible,
- simple à manipuler,
- compatible avec la majorité des technologies web modernes.

JSON est aujourd'hui le format standard utilisé dans les API REST car il facilite la communication entre différentes applications et langages de programmation.

◆ Architecture REST

L'API suit les principes de l'architecture REST (*Representational State Transfer*).

Une API REST permet de communiquer avec le serveur à l'aide de routes appelées endpoints et des méthodes HTTP standards comme :

- GET,
- POST,
- PUT,
- DELETE.

Cette architecture a été choisie car elle permet :

- une meilleure organisation des ressources,

- une communication claire entre le client et le serveur,
- une forte compatibilité avec les applications web modernes,
- une maintenance plus simple de l'application.

REST est aujourd'hui l'architecture la plus utilisée pour les applications web et les services backend modernes.

◆ Liste des endpoints principaux

Un endpoint représente une route de l'API permettant d'accéder à une ressource ou d'effectuer une action spécifique.

Les endpoints doivent être correctement nommés afin :

- d'améliorer la lisibilité de l'API,
- de faciliter la maintenance,
- de respecter les conventions REST,
- de rendre l'API compréhensible pour les développeurs.

Les routes utilisent des noms explicites basés sur les ressources métier de l'application.

◆ Exemple de route API

Méthode	Endpoint	Description
GET	/NomDeLaRoute	Liste
GET	/NomDeLaRoute/{uuid}	Détail
POST	/NomDeLaRoute	Création
PUT	/NomDeLaRoute/{uuid}	Mise à jour
DELETE	/nomDeLaRoute/{uuid}	Suppression

Standardisation des réponses API

Afin de garantir une meilleure cohérence entre les différentes routes de l'API, toutes les réponses retourneront un format standardisé appelé `ApiResponse`.

Cette approche permet :

- d'uniformiser les réponses du backend,

- de faciliter le traitement des données côté frontend Angular,
- d'améliorer la lisibilité des erreurs et des succès,
- de simplifier la maintenance et le débogage de l'application,
- de rendre l'API plus professionnelle et évolutive.

Chaque réponse de l'API contiendra les mêmes champs principaux, qu'il s'agisse d'une réponse réussie ou d'une erreur.

◆ Structure d'une réponse API

Champ	Description
success	Indique si la requête a réussi
code	Code HTTP de la réponse
timestamp	Date et heure de la réponse
message	Message informatif ou erreur
data	Données retournées par l'API

◆ Exemple de réponse succès

```
{
  "success": true,
  "code": 200,
  "timestamp": "2026-05-11T14:30:00",
  "message": "Utilisateur récupéré avec succès",
  "data": {
    "publicId": "8d1b6d93-9f4e-4d5f-9a22-2c8a5d2b7f10",
    "nom": "Dupont",
    "prenom": "Jean"
  }
}
```

◆ Exemple de réponse erreur

```
{
  "success": false,
  "code": 404,
  "timestamp": "2026-05-11T14:35:00",
  "message": "Utilisateur introuvable",
  "data": null
}
```

◆ Pourquoi utiliser un format unique ?

L'utilisation d'un format de réponse unique permet au frontend de toujours savoir comment lire les données retournées par le backend.

Cela facilite notamment :

- la gestion des erreurs,
- l'affichage des messages utilisateurs,
- le traitement des réponses HTTP,
- la maintenance globale de l'application.

Cette approche est largement utilisée dans les architectures backend modernes afin d'améliorer la stabilité et l'évolutivité des API REST.

WebSocket

Afin de permettre la communication en temps réel, l'application utilise WebSocket avec le protocole STOMP.

Contrairement aux requêtes HTTP classiques, WebSocket permet une communication bidirectionnelle permanente entre le client et le serveur sans devoir recharger la page.

Cette technologie est utilisée dans ConnecTeam pour :

- la messagerie instantanée,
- les notifications en temps réel,
- la mise à jour dynamique de certaines informations.

Exemple de point d'accès WebSocket :

/ws/chat

L'utilisation de WebSocket permet d'améliorer l'expérience utilisateur en rendant les échanges plus rapides et interactifs.

Documentation API

Afin de faciliter le développement, les tests et la maintenance de l'application, l'API REST du backend sera documentée avec **Swagger / OpenAPI**.

Cette documentation permettra :

- de visualiser l'ensemble des endpoints disponibles ;
- de tester les requêtes directement depuis le navigateur ;
- de consulter les paramètres et réponses des API ;
- de faciliter l'intégration entre le frontend Angular et le backend Spring Boot.

◆ Technologies utilisées

- SpringDoc OpenAPI
- Swagger UI